

PRAKTIKUM 13

MEDIAPIPE HAND

Nama : Faza Kholish Nurdiansyah
NIM : 234308070
Kelas : TKA 5C
Tautan GitHub: <https://github.com/Faza-13>

ABSTRAK

Praktikum ini membahas implementasi sistem deteksi dan pelacakan tangan secara real-time menggunakan MediaPipe Hand yang diintegrasikan dengan OpenCV. Sistem memanfaatkan webcam sebagai sumber citra untuk mendeteksi keberadaan tangan serta mengekstraksi 21 titik landmark yang merepresentasikan struktur anatomi tangan. Data landmark tersebut digunakan untuk melakukan visualisasi struktur tangan, klasifikasi tangan kiri dan kanan, serta penentuan orientasi telapak tangan (depan dan belakang) berdasarkan hubungan geometris antar titik landmark. Hasil pengujian menunjukkan bahwa sistem mampu mendeteksi tangan dengan baik pada kondisi pencahayaan yang memadai dan latar belakang yang relatif sederhana. Visualisasi landmark berjalan secara stabil dan dapat dimanfaatkan sebagai dasar analisis lanjutan. Meskipun demikian, ditemukan permasalahan pada penentuan orientasi telapak tangan yang terbalik pada salah satu tangan akibat perbedaan orientasi alami antara tangan kiri dan kanan. Permasalahan tersebut dapat diatasi dengan mengombinasikan perhitungan geometris landmark dan informasi klasifikasi jenis tangan. Secara keseluruhan, praktikum ini menunjukkan bahwa pendekatan berbasis landmark dari MediaPipe efektif untuk membangun sistem hand tracking dasar yang dapat dikembangkan lebih lanjut untuk aplikasi interaksi manusia-komputer berbasis gestur.

Kata Kunci: *MediaPipe Hand, OpenCV, hand tracking, landmark tangan, computer vision.*

Pendahuluan

Perkembangan teknologi computer vision mendorong lahirnya sistem interaksi manusia komputer yang lebih alami melalui pengenalan gestur tangan sebagai media input non-kontak. Pendekatan ini memungkinkan pengguna berinteraksi dengan sistem tanpa perangkat fisik tambahan, sehingga lebih intuitif dan fleksibel untuk berbagai aplikasi interaktif (Author dkk., 2025).

Teknologi computer vision memanfaatkan citra atau video untuk mengekstraksi informasi visual dan mengenali objek secara otomatis, termasuk bagian tubuh manusia seperti tangan. Penerapan computer vision pada pengenalan

gestur tangan memungkinkan komputer mengenali pola gerakan jari dan telapak tangan secara real-time sebagai bentuk perintah atau sinyal tertentu (Prasetya dkk., 2026).

Framework MediaPipe banyak digunakan dalam pengembangan sistem pelacakan tangan karena mampu mendeteksi hingga 21 titik landmark tangan secara real-time dengan tingkat akurasi yang baik. Landmark tersebut merepresentasikan struktur anatomi tangan dan digunakan sebagai dasar untuk mengenali posisi jari, orientasi tangan, serta jenis gestur yang dilakukan pengguna (Author dkk., 2025; Prasetya dkk., 2026).

Pemanfaatan OpenCV bersama MediaPipe mempermudah proses pengambilan citra, pemrosesan frame video, serta visualisasi hasil deteksi landmark tangan pada layar. Kombinasi kedua pustaka ini terbukti efektif dalam berbagai penelitian untuk membangun sistem pengenalan gestur tangan secara real time dan interaktif (Author dkk., 2025; Maliana dkk., 2025).

Namun, beberapa penelitian menunjukkan bahwa orientasi tangan sangat memengaruhi keberhasilan deteksi landmark. Gestur dengan posisi telapak tangan menghadap kamera cenderung lebih mudah dikenali dibandingkan gestur dengan orientasi punggung tangan, karena sebagian landmark tertutup dan sulit terdeteksi secara optimal (Salsabila dkk., 2025).

Penerapan sistem pengenalan gestur tangan telah digunakan dalam berbagai bidang, seperti permainan interaktif, sistem keamanan berbasis sinyal bantuan, serta pengendalian robot lengan. Hal ini menunjukkan bahwa teknologi pengenalan gestur tangan memiliki potensi besar untuk dikembangkan lebih lanjut dalam aplikasi nyata yang membutuhkan interaksi manusia dan komputer secara natural dan bebas sentuh (Maliana dkk., 2025).

Tujuan

1. Mampu mengimplementasikan deteksi tangan secara real-time menggunakan MediaPipe dan OpenCV.
2. Mampu mengidentifikasi tangan kiri dan kanan berdasarkan data klasifikasi.

3. Mampu menentukan posisi orientasi tangan (depan atau belakang) menggunakan analisis koordinat landmark.
4. Memahami konsep dasar pengolahan citra dan geometri sederhana dalam computer vision.

Manfaat

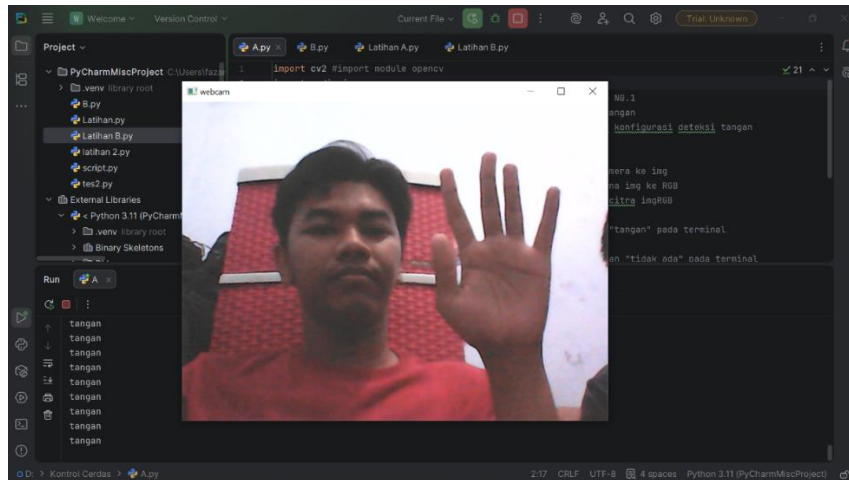
1. Memberikan pemahaman praktis mengenai penerapan visi komputer untuk mendeteksi dan melacak tangan secara real-time menggunakan pendekatan berbasis landmark.
2. Membantu mahasiswa memahami konsep interaksi manusia dan komputer berbasis gestur sebagai alternatif input yang lebih natural dibandingkan perangkat konvensional.
3. Menjadi dasar pengembangan aplikasi interaktif berbasis gestur tangan, seperti permainan atau sistem kendali sederhana.
4. Memberikan wawasan mengenai pemanfaatan gestur tangan dalam sistem keamanan atau sistem bantuan berbasis isyarat tangan.
5. Menjadi landasan awal untuk pengembangan sistem pengendalian perangkat atau robot menggunakan gestur tangan sebagai antarmuka yang lebih intuitif.

Hasil Program

Percobaan A (mendeteksi tangan)

```
1 import cv2 #import module opencv
2 import mediapipe
3 capture = cv2.VideoCapture(0) #video capture pada device kamera NO.1
4 mediapipehand = mediapipe.solutions.hands #inisiliasi deteksi tangan
5 tangan = mediapipehand.Hands() #variable tangan untuk menyimpan konfigurasi deteksi tangan
6
7 while True:
8     success, img = capture.read() #menyimpan citra tangkapan kamera ke img
9     imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) #mengubah warna img ke RGB
10    result = tangan.process(imgRGB) #melakukan pemrosesan dari citra imgRGB
11    if result.multi_hand_landmarks:
12        print("tangan") #ketikan tangan terdeteksi menampilkan "tangan" pada terminal
13    else:
14        print("tidak ada") #ketikan tangan terdeteksi menampilkan "tidak ada" pada terminal
15    cv2.imshow("webcam", img)
16    cv2.waitKey(10)
17    if cv2.waitKey(10) & 0xFF == ord('q'):
18        break
19 capture.release() #Tutup webcam dan jendela tampilan saat 1 ditekan
20 cv2.destroyAllWindows()
```

Hasil



Percobaan B (mendeteksi tangan dengan titik landmarks)

```
1 import cv2 #import module opencv
2 import mediapipe
3 capture = cv2.VideoCapture(0)
4 mediapipehand = mediapipe.solutions.hands
5 tangan = mediapipehand.Hands(max_num_hands=1)
6 mpdraw = mediapipe.solutions.drawing_utils
7
8 while True:
9     success, img = capture.read() #membaca frame video
10    imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
11    results = tangan.process(imgRGB)
12    if results.multi_hand_landmarks:
13        for titiktangan in results.multi_hand_landmarks:
14            mpdraw.draw_landmarks(img, titiktangan, mediapipehand.HAND_CONNECTIONS)
15            for id, titik in enumerate(titiktangan.landmark):
16                print(id)
17                print(titik.x)
18                print(titik.y)
19    cv2.imshow('webcam', img)
20    cv2.waitKey(10)
21    if cv2.waitKey(10) & 0xFF == ord('q'):
22        break # Tutup webcam dan jendela tampilan saat q ditekan
23 cap.release()
24 cv2.destroyAllWindows()
25
```

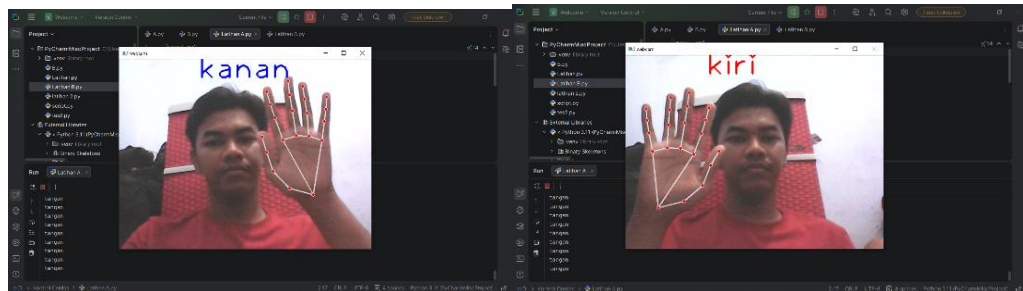
Hasil



Percobaan latihan A (mendeteksi tangan kanan dan kiri)

```
1  import cv2
2  import mediapipe
3
4  capture = cv2.VideoCapture(0)
5
6  mediapipehand = mediapipe.solutions.hands
7  tangan = mediapipehand.Hands(max_num_hands=2)
8  mpdraw = mediapipe.solutions.drawing_utils
9
10 while True:
11     success, img = capture.read()
12     if not success:
13         print("loop jalan")
14         break
15     imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
16     results = tangan.process(imgRGB)
17     if results.multi_hand_landmarks:
18         print("tangan")
19         for titiktangan in results.multi_hand_landmarks:
20             mpdraw.draw_landmarks(img, titiktangan, mediapipehand.HAND_CONNECTIONS)
21         if results.multi_handedness:
22             for idx, hand in enumerate(results.multi_handedness):
23                 if hand.classification[0]:
24                     if hand.classification[0].index == 1: #klasifikasi tangan kanan
25                         cv2.putText(img, "kanan", (200, 50), cv2.FONT_HERSHEY_PLAIN, 5, (255, 0, 0), 3) #memberika
26                     elif hand.classification[0].index == 0: #klasifikasi tangan kiri
27                         cv2.putText(img, "kiri", (200, 50), cv2.FONT_HERSHEY_PLAIN, 5, (0, 0, 255), 3)
28
29     else:
30         print("tidak ada")
31         cv2.imshow("webcam", img)
32         if cv2.waitKey(1) & 0xFF == ord('q'):
33             break
34     capture.release()
35     cv2.destroyAllWindows()
```

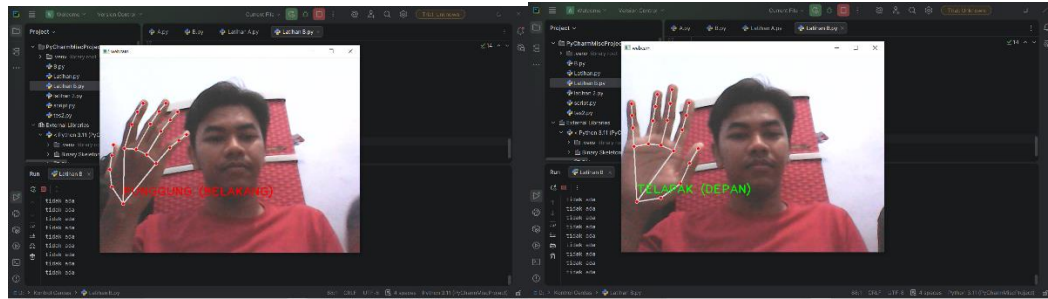
Hasil



Percobaan latihan B (mendeteksi bagian telapak tangan dan punggung tangan)

```
1 import cv2
2 import mediapipe
3
4 capture = cv2.VideoCapture(0)
5
6 mediapipehand = mediapipe.solutions.hands
7 tangan = mediapipehand.Hands(max_num_hands=2)
8 mpdraw = mediapipe.solutions.drawing_utils
9
10 while True:
11     success, img = capture.read()
12     if not success:
13         break
14
15     imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
16     results = tangan.process(imgRGB)
17
18     if results.multi_hand_landmarks and results.multi_handedness:
19         for idx, titik tangan in enumerate(results.multi_hand_landmarks):
20             mpdraw.draw_landmarks(img, titik tangan, mediapipehand.HAND_CONNECTIONS)
21
22             h, w, c = img.shape
23
24             wrist = titik tangan.Landmark[0]
25             index_mcp = titik tangan.Landmark[5]
26             pinky_mcp = titik tangan.Landmark[17]
27
28             x0, y0 = int(wrist.x * w), int(wrist.y * h)
29             x5, y5 = int(index_mcp.x * w), int(index_mcp.y * h)
30             x17, y17 = int(pinky_mcp.x * w), int(pinky_mcp.y * h)
31
32             cross = (x5 - x0) * (y17 - y0) - (y5 - y0) * (x17 - x0)
33
34             # Ambil label tangan
35             label = results.multi_handedness[idx].classification[0].label
36
37             if label == "Right":
38                 kondisi = cross > 0
39             else: # Left
40                 kondisi = cross < 0
41
42             if kondisi:
43                 posisi = "TELAPAK (DEPAN)"
44                 warna = (0, 255, 0)
45             else:
46                 posisi = "PUNGGUNG (BELAKANG)"
47                 warna = (0, 0, 255)
48
49             cv2.putText(img, posisi,
50                         (x0, y0 - 20),
51                         cv2.FONT_HERSHEY_PLAIN,
52                         2,
53                         warna,
54                         2)
55
56             else:
57                 print("tidak ada")
58
59             cv2.imshow("webcam", img)
60
61             if cv2.waitKey(1) & 0xFF == ord('q'):
62                 break
63
64             capture.release()
65             cv2.destroyAllWindows()
```

Hasil



Analisis

Pada percobaan kali ini dilakukan implementasi sistem deteksi dan pelacakan tangan menggunakan MediaPipe Hand yang diintegrasikan dengan OpenCV. Pada bagian awal program, fungsi `cv2.VideoCapture(0)` digunakan untuk mengakses webcam sebagai sumber citra, sedangkan `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` berfungsi mengubah format warna dari BGR ke RGB agar sesuai dengan format input yang dibutuhkan oleh MediaPipe. Objek `Hands()` pada MediaPipe digunakan untuk menginisialisasi model deteksi tangan yang akan memproses setiap frame video secara real-time.

Pada percobaan A, tujuan utama adalah mendeteksi keberadaan tangan. Fungsi `tangan.process(imgRGB)` berperan untuk memproses citra hasil tangkapan webcam dan menghasilkan objek hasil deteksi yang berisi informasi apakah tangan terdeteksi atau tidak. Kondisi `if results.multi_hand_landmarks:` digunakan sebagai logika utama untuk menentukan apakah terdapat tangan pada frame yang diproses. Apabila bernilai benar, sistem menampilkan bahwa tangan terdeteksi, sedangkan jika bernilai salah, sistem menyatakan tidak ada tangan. Bagian ini menunjukkan mekanisme dasar deteksi objek menggunakan MediaPipe.

Pada percobaan B, sistem dikembangkan untuk menampilkan landmark tangan pada layar. Objek `mpdraw.draw_landmarks(img, titiktangan, mediapipehand.HAND_CONNECTIONS)` berfungsi menggambar titik-titik landmark beserta koneksinya pada citra asli sehingga struktur tangan dapat divisualisasikan secara langsung. Landmark yang dihasilkan berupa koordinat ternormalisasi (x, y, z) yang dapat dimanfaatkan untuk analisis lebih lanjut. Visualisasi ini membantu memverifikasi bahwa MediaPipe tidak hanya mendeteksi tangan, tetapi juga memetakan struktur anatomi tangan secara detail.

Pada percobaan latihan A, dilakukan klasifikasi tangan kiri dan kanan. Informasi klasifikasi diperoleh dari `results.multi_handedness` yang berisi label jenis tangan untuk setiap tangan yang terdeteksi. Pengkondisian nilai indeks klasifikasi digunakan untuk menentukan apakah tangan yang terdeteksi merupakan tangan kiri atau kanan, kemudian fungsi `cv2.putText()` digunakan untuk menampilkan teks keterangan “kiri” atau “kanan” pada layar. Bagian ini menunjukkan bagaimana

metadata hasil deteksi MediaPipe dapat dimanfaatkan untuk menambah informasi semantik pada hasil visualisasi.

Pada percobaan latihan B, dilakukan penentuan orientasi tangan (telapak menghadap kamera atau membelakangi kamera) dengan memanfaatkan hubungan geometris antar landmark utama, seperti pergelangan tangan serta pangkal jari telunjuk dan kelingking. Landmark-landmark tersebut digunakan sebagai titik acuan untuk membentuk vektor arah tangan, kemudian dihitung arah relatifnya untuk menentukan orientasi telapak tangan. Logika pengambilan keputusan pada bagian ini menjadi inti dari proses klasifikasi orientasi tangan, sementara fungsi `cv2.putText()` kembali digunakan untuk menampilkan hasil klasifikasi orientasi pada layar. Percobaan ini menunjukkan bahwa data landmark dapat dimanfaatkan tidak hanya untuk visualisasi, tetapi juga untuk analisis spasial yang lebih kompleks.

Kesimpulan

Berdasarkan percobaan yang telah dilakukan dan hasil yang telah didapat maka dapat disimpulkan bahwa:

1. MediaPipe Hand mampu melakukan deteksi dan pelacakan tangan secara real-time dengan cukup stabil menggunakan representasi 21 titik landmark.
2. Integrasi MediaPipe dengan OpenCV mempermudah proses pengambilan citra dari webcam, pemrosesan frame video, serta visualisasi hasil deteksi landmark pada layar.
3. Sistem berhasil membedakan kondisi ada atau tidak adanya tangan di depan kamera pada kondisi pencahayaan yang memadai.
4. Informasi landmark dapat dimanfaatkan untuk analisis lanjutan, seperti klasifikasi tangan kiri dan kanan serta penentuan orientasi telapak tangan (depan dan belakang).
5. Penentuan orientasi telapak tangan memungkinkan dilakukan menggunakan hubungan geometris antar landmark, namun hasil awal dapat terbalik pada salah satu tangan sehingga memerlukan penyesuaian logika berdasarkan klasifikasi tangan kiri dan kanan.

Saran

Untuk pengembangan selanjutnya, sistem deteksi tangan disarankan untuk diuji pada kondisi pencahayaan yang lebih bervariasi, jarak kamera yang berbeda, serta latar belakang yang lebih kompleks agar diketahui batasan performa deteksi secara lebih menyeluruh. Penentuan orientasi telapak tangan dapat ditingkatkan akurasi dengan menggunakan kombinasi beberapa landmark sebagai titik acuan serta mempertimbangkan klasifikasi tangan kiri dan kanan secara bersamaan. Selain itu, pengembangan lanjutan dapat diarahkan pada pengenalan gestur yang lebih kompleks dengan memanfaatkan metode pembelajaran mesin atau deep learning, serta mengintegrasikan sistem ke dalam aplikasi nyata seperti permainan interaktif, kontrol objek virtual, atau pengendalian perangkat sederhana agar hasil praktikum memiliki nilai aplikatif yang lebih tinggi.

REFERENSI

- Author, A. R. N., Putra, R. M. A., Seran, S. H., & Sahertian, J. (2025). *Deteksi Kode Signal For Help Pada Gestur Tangan Menggunakan OpenCV*. 4.
- Maliana, D. G., Dewangga, R. A., & Saputra, M. A. (2025). *Implementasi Citra Gestur Tangan Pada Permainan Rock Paper Scissor Menggunakan OpenCV Dan Mediapipe*. 4.
- Prasetya, A. Y., Subairi, & Rabi, A. (2026). *Deteksi Gestur Tangan Menggunakan Computer Vision Untuk Pengendalian Robot Lengan*.
- Salsabila, Z. N., Adnan, M. M., Rusli, B. D., & Sihombing, D. (2025). *Volume 6, Nomor 4, Desember 2025*. 6.